NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

CCDS25012- When planning meets controlling

Presented by Chou Yu-Fan

Supervised by Asst Prof Sung Yoonchang

1. Motivation

This project addresses the problem of **real-time safe navigation for robots in dynamic obstacle environments**. Specifically, we aim to integrate pybullet_planning with Control Barrier Function QP (CBF-QP) to ensure that a robot can track a planned path while provably avoiding moving obstacles at runtime.

Pybullet_planning provides collision-free path planning assuming a **static environment**. Once a path is executed, there is no runtime mechanism to handle **moving obstacles** — safety is only guaranteed at planning time, not at execution time. CBF-QP addresses exactly this gap by providing a formal, real-time safety certificate during control.

2. Methodology

Path Planning: RRT generates a nominal path.

Replan trigger: When a new obstacle spawns on the current path, replan is triggered.

Gap problem: Even we assume replanning takes place immediately, the robot continues moving due to inertia. The physical constraint may result in collision when trying to switch to the new path.

CBF-QP as safety net: CBF-QP modifies the control input in real-time to maintain a safe distance from the new obstacle.

3. Findings

Let the states be:

$$\boldsymbol{u} = [\boldsymbol{a}, \boldsymbol{\alpha}]^{\boldsymbol{T}}, \boldsymbol{x} = \left[\boldsymbol{p_x}, \boldsymbol{p_y}, \boldsymbol{\theta}, \boldsymbol{v}, \boldsymbol{\omega}\right]^{\boldsymbol{T}}$$

Calculate the first derivative of h (Lie derivative)

$$\dot{\boldsymbol{h}} = \boldsymbol{2\delta_x \dot{p}_x + 2\delta_y \dot{p}_y = 2v\left(\delta_x cos\theta + \delta\delta_y sin\theta\right)}$$

Since the control input does not appear in the first derivative of h, a second-order CBF formulation is required.

$$\ddot{\boldsymbol{h}} = \boldsymbol{d/dt}\left[\boldsymbol{2v\left(\delta_x cos\theta + \delta_y sin\theta\right)}\right]$$

After simplification:

$$\ddot{\boldsymbol{h}} = \boldsymbol{2a\left(\delta_x cos\theta + \delta_y sin\theta\right)}$$
$$\boldsymbol{+2v\left[v + \omega\left(\delta_y cos\theta - \delta_x sin\theta\right)\right]}$$

General constraint is

$$\ddot{\boldsymbol{h}} + \boldsymbol{\gamma_1 \dot{h} + \gamma_2 \psi_1 \geq 0}$$

Substituting in our first and second order lie derivatives; we have our final constraint.

$$\boldsymbol{2\left(\delta_x cos\theta + \delta_y sin\theta\right) \cdot a \geq}$$
$$\boldsymbol{-2v\left[v + \omega\left(\delta_y cos\theta - \delta_x sin\theta\right)\right] - \gamma_1 \dot{h} - \gamma_2 \psi_1}$$

vid1	vid2	vid3

Neglecting the physical aspect, RRT will always work unless the robot is surrounded, as shown in **vid1**. This is because it can control its position at any moment.

Taking physics into account, **inertia** is a critical factor. No matter how hard the brakes are engaged, a collision is possible if the obstacle is too close, as seen in Video 2.

In order to prevent collision, the following methods were employed. First, the planner generates a nominal route. However, when entering a dangerous zone ($h < 0$, when too close to the obstacle), it triggers CBF-QP, which attempts to steer the robot away. When escaping, it may collide with another existing obstacle, the controller must continuously monitor the nearest obstacle. The result is shown on vid3.

4. Conclusions

This project demonstrates that integrating CBF-QP with RRT-based path planning provides a robust framework for real-time safe navigation in dynamic environments. While RRT alone fails to account for inertia and runtime obstacle emergence, CBF-QP acts as a formal safety filter that provably modifies control inputs to prevent collisions. The two-layer architecture — planning-time nominal path + runtime safety guarantee — proves effective in simulation, with the robot successfully avoiding dynamic obstacles even under physical constraints.

5. Future work

Several directions remain open for future exploration. First, the current implementation always tracks the single nearest obstacle; extending this to handle **multiple simultaneous obstacles** with priority weighting would improve robustness. Second, replacing RRT with a more informed planner (e.g., **RRT* or trajectory optimization**) could reduce the frequency of CBF interventions. Third, validating the framework on a **physical robot platform** (beyond PyBullet simulation) would confirm real-world transferability. Finally, exploring **adaptive CBF parameters** that adjust safety margins based on obstacle velocity could improve performance in high-speed dynamic scenarios.